

Lesson 5 — Intro to WPILib



  RamRodz Robotics · Team 6366 · Programming Worksheet

Name

Date

  RAM RODZ ROBOTICS | TEAM #6366 | HILLCREST & WOODMONT HIGH SCHOOLS ·
SIMPSONVILLE, SC

 Focus Check-In:

 Struggling

 Okay

 Ready

TODAY'S BIG QUESTION

The robot has motors and a controller — but how does your code actually connect the two so that pushing a stick moves a wheel?

 Parking Lot — Random Thoughts & Questions

1

Key Words — Check Off When Explained



WPILib

The official Java library that FRC teams use to control robot hardware. It gives you ready-made classes for motors, controllers, and sensors.

PWM

Pulse Width Modulation — a simple signal type used by basic motor controllers. They plug into numbered PWM ports (0–9) on the roboRIO.

import

A Java keyword that loads a class from a library into your file so you can use it. Without an import, Java doesn't know the class exists.

CAN Bus

Controller Area Network — a communication system that lets smart motor controllers like the TalonFX talk to the roboRIO over a shared wire loop.

XboxController

A WPILib class that represents the Xbox controller plugged into the driver station. You read buttons and joystick axes from it.

Motor Controller

The electronics board between the robot's battery and a motor. Your code tells the motor controller what speed to run, and it handles the power.

2

What is WPILib?



Listen — Instructor explains what WPILib is and why FRC teams use it.

WPILib is a giant toolbox of pre-built Java classes written specifically for FRC robots. Instead of writing code from scratch to talk to a motor or read a joystick, you use WPILib's classes — they handle the low-level electronics for you.

To use a WPILib class in your file, you add an **import** statement at the very top — one line per class. This tells Java where to find the class definition.

Java — Import Statements

```
// These lines go at the TOP of your file, before the class declaration

import edu.wpi.first.wpilibj.XboxController;    // driver controller
import edu.wpi.first.wpilibj.motorcontrol.PWMSparkMax; // PWM motor controller
import com.ctre.phoenix6.hardware.TalonFX;      // CAN motor controller
```

💬 Plain English: **import tells Java "I need that class — go find it in the library."** Without the import, Java sees `XboxController` and has no idea what it is. Every WPILib class you use needs its own import line at the top.

Anatomy of an Import Statement

```
import  edu.wpi.first.wpilibj.XboxController;
  ↑      ↑              ↑
keyword package path      class name
```

 **Quick Check — What would you change to import `PWMSparkMax` instead?**

☐ I understand that import statements must go at the top of the file, one per class.

3 The XboxController

☐ Listen — Instructor explains how to create an `XboxController` object and read from it.

An **XboxController** is a WPILib class that represents the physical Xbox controller plugged into the driver station. You create one object — then call its methods to read button presses and joystick positions.

The constructor takes one number: the **port** the controller is connected to on the driver station (0 for the first controller, 1 for the second).

Java — Creating and Reading an XboxController

```
// Declare the controller as a field (inside the class, outside any method)
XboxController controller = new XboxController(0); // port 0 = driver controller

// — Inside a method (like teleopPeriodic) —————

// Read the left joystick's Y axis (push forward = negative, pull back = positive)
double stickValue = controller.getLeftY(); // returns -1.0 to 1.0

// Read whether the A button is currently held down
boolean aPressed = controller.getAButton(); // true if pressed, false if not

// Read the right joystick's Y axis
double rightStick = controller.getRightY(); // returns -1.0 to 1.0
```

💡 Plain English: **Joystick axes return a decimal from -1.0 (full one direction) to 1.0 (full the other).** At rest they return 0.0. Buttons return true when held and false when released. You can feed those values straight into a motor.

Method	Returns	What It Reads
<code>getLeftY()</code>	double	Left joystick, up/down axis
<code>getLeftX()</code>	double	Left joystick, left/right axis
<code>getRightY()</code>	double	Right joystick, up/down axis
<code>getRightX()</code>	double	Right joystick, left/right axis
<code>getAButton()</code>	boolean	A button
<code>getBButton()</code>	boolean	B button
<code>getLeftBumper()</code>	boolean	Left bumper (LB)
<code>getRightBumper()</code>	boolean	Right bumper (RB)

 Try It — Write the lines to read these two inputs from a controller named 'driver'

Store the right joystick Y axis in a double named `turnSpeed`:

Store whether the B button is pressed in a boolean named `fireButton`:

☐ I filled in both write-in lines and understand what each controller method returns.

4 Driving a Motor — PWMSparkMax



☐ Listen — Instructor explains PWM motor controllers and shows how `.set()` works.



PWMSparkMax

The SPARK MAX (by REV Robotics) is one of the most common motor controllers in FRC. When wired to a PWM port on the roboRIO, you control it with WPILib's `PWMSparkMax` class. The constructor takes the PWM port number (printed on the roboRIO, 0–9).

Java — Creating and Using a PWMSparkMax

```
// Declare as a field — inside the class, outside any method
PWMSparkMax driveMotor = new PWMSparkMax(0); // connected to PWM port 0

// — Inside teleopPeriodic —————

// Set the motor to 50% forward power
driveMotor.set(0.5);

// Set the motor to full reverse
driveMotor.set(-1.0);

// Stop the motor
driveMotor.set(0.0);
```

💡 Plain English: **.set()** takes a number from **-1.0 to 1.0**. Positive = forward, negative = reverse, zero = stop. Pass it directly the value you read from a joystick axis and the motor follows the stick.

Connecting the controller to the motor — this is the key pattern:

Java — Controller → Motor (the full data flow)

```
XboxController driver = new XboxController(0); // controller on port 0
PWMSparkMax leftDrive = new PWMSparkMax(0); // motor on PWM port 0

// — Called repeatedly during teleop —————
double speed = driver.getLeftY(); // 1. read the stick (-1.0 to 1.0)
leftDrive.set(speed);           // 2. send that value straight to the motor
```

💡 Plain English: **Two lines — read, then send**. The joystick axis already returns -1.0 to 1.0, which is exactly what .set() expects. No math needed. Push the stick forward, the motor spins forward. Pull back, it reverses.

✏ Try It — Write the two lines to connect a right-stick axis to a motor named 'rightDrive'

☐ I wrote the two lines above and understand the read → send pattern.

5 The TalonFX — CAN Bus Motors

☐ Listen — Instructor explains what CAN bus is and how TalonFX differs from PWMSparkMax.



TalonFX (CTRE)

The TalonFX motor controller is built into the **Falcon 500** and **Kraken X60** motors — two of the most powerful motors used in FRC. Unlike PWM controllers, the TalonFX communicates over the **CAN bus** and is programmed using CTRE's **Phoenix 6** vendor library, which you add to your project separately from WPILib.

PWM vs CAN — what's the difference?

	PWM (PWMSparkMax)	CAN Bus (TalonFX)
Connection	PWM port on roboRIO (wire per motor)	CAN bus loop shared by all CAN devices
Identifier	Port number (0–9)	Device ID (set in Phoenix Tuner)
Library	WPILib (built-in)	Phoenix 6 (vendor library, added separately)
Features	Basic speed control	Advanced control, position, velocity, current limits

Java — Importing and Creating a TalonFX

```
// Phoenix 6 import — requires the Phoenix 6 vendor library in your project
import com.ctre.phoenix6.hardware.TalonFX;

// — Inside your robot class —————

// The TalonFX constructor takes a device ID — NOT a PWM port number
// Device IDs are configured in Phoenix Tuner X software
TalonFX shooterMotor = new TalonFX(1); // device ID 1 on the CAN bus
```

💡 Plain English: **Creating a TalonFX looks just like creating a PWMSparkMax — new ClassName(id) — but the number is a CAN device ID, not a port.** Device IDs are set using a tool called Phoenix Tuner X, which talks to the motor controller over USB or the network. The full Phoenix 6 API for controlling the motor is a future lesson — for now, focus on creating the object correctly.

 **Why does the device ID matter?** Every device on the CAN bus shares the same two wires. The device ID is how the roboRIO tells one TalonFX from another. If two devices share the same ID, your robot won't behave predictably. Always verify device IDs in Phoenix Tuner X before running code.

 Try It — Fill in the blanks

Create a TalonFX object named `climbMotor` with device ID 3:

Write the import statement you would need for the TalonFX class:

In one sentence: what is the difference between a PWM port number and a CAN device ID?

☐ I filled in all three write-in answers and understand the PWM vs CAN difference.

 Robot Scenario — Your Turn



 Scenario

Your team is wiring up the robot for the first time this season. The drivetrain uses two **PWMSparkMax** controllers (port 0 = left, port 1 = right). The shooter uses a **TalonFX** (device ID 2). The drive team will use a single **XboxController** on port 0.

- Left drive follows the **left stick Y axis**
- Right drive follows the **right stick Y axis**
- The **A button** fires the shooter (TalonFX) — for now, just create the object; the full Phoenix 6 control call comes in a later lesson

Write the field declarations and the controller-to-motor wiring lines below.



Write the three import statements you need



Declare the four hardware objects as fields



Write the teleop wiring — read controller inputs and send to the drive motors

☐

I completed all three sections of the scenario — imports, field declarations, and teleop wiring.

☒ **End of Lesson — I Can... (click each one when you're confident)**

☐ Write an import statement for any WPILib or Phoenix 6 class and explain what it does.

☐ Create an XboxController object and use it to read a joystick axis and a button.

☐ Create a PWMSparkMax object, pass it a port number, and use `.set()` to control its speed.

☐ Create a TalonFX object with a CAN device ID and explain how CAN differs from PWM.

☐ Wire a joystick axis to a motor in two lines — read the stick, send the value to `.set()`.